International Journal of Heat and Mass Transfer 201 (2022) 123649

Contents lists available at ScienceDirect

International Journal of Heat and Mass Transfer

journal homepage: www.elsevier.com/locate/hmt

Multi-GPU thermal lattice Boltzmann simulations using OpenACC and MPI

Ao Xu[a,b,c,*], Bo-Tao Li[a]

[a] School of Aeronautics, Northwestern Polytechnical University, Xi'an 710072, China
[b] Institute of Extreme Mechanics, Northwestern Polytechnical University, Xi'an 710072, China
[c] Key Laboratory of Icing and Anti/De-icing, China Aerodynamics Research and Development Center, Mianyang 621000, China

ARTICLE INFO

Article history:
Received 27 September 2022
Revised 6 November 2022
Accepted 7 November 2022

Keywords:
Lattice Boltzmann method
Thermal convective flows
GPU
OpenACC
MPI

ABSTRACT

We assess the performance of the hybrid Open Accelerator (OpenACC) and Message Passing Interface (MPI) approach for multi-graphics processing units (GPUs) accelerated thermal lattice Boltzmann (LB) simulation. The OpenACC accelerates computation on a single GPU, and the MPI synchronizes the information between multiple GPUs. With a single GPU, the two-dimension (2D) simulation achieved 1.93 billion lattice updates per second (GLUPS) with a grid number of 8193^2, and the three-dimension (3D) simulation achieved 1.04 GLUPS with a grid number of 385^3, which is more than 76% of the theoretical maximum performance. On multi-GPUs, we adopt block partitioning, overlapping communications with computations, and concurrent computation to optimize parallel efficiency. We show that in the strong scaling test, using 16 GPUs, the 2D simulation achieved 30.42 GLUPS and the 3D simulation achieved 14.52 GLUPS. In the weak scaling test, the parallel efficiency remains above 99% up to 16 GPUs. Our results demonstrated that, with improved data and task management, the hybrid OpenACC and MPI technique is promising for thermal LB simulation on multi-GPUs.

1. Introduction

Over the past half-century, the development of semiconductor transistors has driven rapid growth and prosperity of high-performance computing (HPC). The miniaturization of computer components was foreseen by physicist Richard Feynman in his 1959 address "There is Plenty of Room at the Bottom" [1]. Later in 1975, Gordon Moore, the founder of Intel Corporation, predicted that the number of transistors per computer chip would double every two years, which is known as Moore's law [2]. This trend held up considerably well until recently when transistors reduce their physical size and are reaching the atomic scale, implying that Moore's law is nearing its end and is anticipated to flatten by 2025 [3]. Alternative avenues for growth in computer performance include hardware architecture, software, and algorithms [4]. In the post-Moore era, computer architects should focus on hardware streamlining and provide additional chip area for more circuitry to operate in parallel, rather than use more transistors and increase the complexity of processing cores as they used to do. For example, the graphics process unit (GPU) contains many parallel lanes and it can exploit much more parallelism, thus it delivers much more performance on computations.

The lattice Boltzmann (LB) method is a numerical approach to simulate fluid flows and associate heat and mass transfer processes [5,6]. Specifically, the LB method describes the evolution of particle density distribution, which originated from the Boltzmann kinetic theory but practically reflects hydrodynamic behavior at the continuum scale. Mesoscopic physical pictures can be easily incorporated into the LB method, and macroscopic physical conservation laws can be recovered with a relatively low computational cost. After three decades of development of the LB method, the computational fluid dynamics community has witnessed its powerful ability to simulate complex flows, such as gas-liquid two-phase flow [7,8], particulate flow [9–11], fluid-structure interaction [12], flow in porous media [13], and so on. Advancements in HPC utilizing heterogeneous architecture, namely the combined traditional central processing unit (CPU) and the emerging accelerators (such as GPUs), further facilitate the application of LB simulations in large-scale engineering problems [14,15]. Open-source codes based on the LB method, including OpenLB [16], Palabos [17], and Sailfish [18], even aim to brace the forthcoming Exascale supercomputing [19]. Review articles on LB simulation using GPUs can be found by Navarro-Hinojosa et al. [20], Niemeyer and Sung [21].

* Corresponding author.
E-mail address: axu@nwpu.edu.cn (A. Xu).

https://doi.org/10.1016/j.ijheatmasstransfer.2022.123649
0017-9310/

The pioneer works implementing LB simulations on graphics hardware were achieved by using textures and render buffers [22]. Since 2007, with the introduction of Compute Unified Device Architecture (CUDA), which is a parallel computing platform and application programming interface (API), significant advances in applying LB simulation on GPUs have been made [23–26]. Meanwhile, the Open Computing Language (OpenCL, initially released in 2009), which is an open standard for writing programs executing across the heterogeneous platform, has also accelerated the LB simulation without being restricted to only running on NVIDIA's hardware. For example, Sailfish, an open-source LB solver is built dynamically at run-time in CUDA or OpenCL [18]. However, these two APIs require substantial changes in the original code, thus threatening the code's correctness, portability, and maintainability. The third accelerating approach is using Open Accelerators (OpenACC), which is a platform-independent, high level and directive-based programming standard. The programmers provide hints as annotations to the original code, specifying that a certain loop should run in parallel on the accelerator, then compute-intensive calculations are offloaded to the accelerator device without the need to explicitly manage data transfers between the host and the accelerator. Recently, Open Multi-Processing (OpenMP) support for GPUs is also available. Both OpenACC and OpenMP are directive based; the difference is that OpenACC is more descriptive, while OpenMP is more prescriptive (using distribute constructs to explicitly maps work-loads) [27].

So far, there are fewer implementations of LB code using OpenACC compared to that using CUDA, primarily due to concerns regarding computational efficiency [27–29]. Xu et al. [30] demonstrated that for thermal flows (e.g., fluid flows and heat transfer in the side heated cavity), a speedup of around 53X can be achieved when using OpenACC acceleration compared with the serial code. The results were obtained on a single Tesla K20c GPU, and 282.6 million lattice update per second (MLUPS, defined later in this paper) were reached for double-precision floating calculation. Although higher performance could be gained using single-precision floating calculation [31], it would pose threats to the accuracy of the simulation results, particularly for turbulent flow simulations [32].

To utilize the computing power of multi-node GPU clusters, LB implementations based on a hybrid OpenACC and Message Passing Interface (MPI) approach can be used for massively parallel simulations of problems with larger domain sizes. However, simulations running on multiple GPUs have to face Peripheral Component Interconnect Express (PCI-e) bottlenecks and minimize inter-GPU communications. In this work, we address implementation issues when using hybrid MPI and OpenACC to accelerate LB simulations. We show that ultra-high computational performance can be achieved with proper implementations. The rest of this paper is organized as follows. In Section 2, we introduce numerical details for the simulation of thermal convection, including the mathematical model and the corresponding LB model. In Section 3, we present details for implementation and optimization of the hybrid OpenACC and MPI, as well the parallel performance measured via the strong scaling test. In Section 4, we present parallel performance measured via the weak scaling test. In Section 5, the main findings of this work are summarized.

2. Numerical method

2.1. Mathematical model for thermal convection

We simulate thermal convection based on the Boussinesq approximation. We assume the fluid flow is incompressible, and we treat the temperature as an active scalar that influences the velocity field through the buoyancy. The viscous heat dissipation and compression work are neglected, and all the transport coefficients are assumed to be constants. Then, the governing equations can be written as

$$\nabla \cdot \mathbf{u} = 0 \tag{1a}$$

$$\frac{\partial \mathbf{u}}{\partial t} + \mathbf{u}\cdot\nabla\mathbf{u} = -\frac{1}{\rho_0}\nabla P + \nu\nabla^2\mathbf{u} + g\beta_T(T-T_0)\hat{\mathbf{z}} \tag{1b}$$

$$\frac{\partial T}{\partial t} + \mathbf{u}\cdot\nabla T = \alpha_T\nabla^2 T \tag{1c}$$

where $\mathbf{u}$, P and T are the velocity, pressure, and temperature of the fluid, respectively. ρ_0 and T_0 are reference density and temperature, respectively. ν, β_T and α_T denote the viscosity, thermal expansion coefficient, and thermal diffusivity of the fluid, respectively. $\hat{\mathbf{z}}$ is the unit parallel to gravity. With the scaling

$$\begin{aligned} &\mathbf{x}^* = \mathbf{x}/H, \quad t^* = t/\sqrt{H/(\beta_T g\Delta_T)}, \quad \mathbf{u}^* = \mathbf{u}/\sqrt{\beta_T g H\Delta_T}, \\ &P^* = P/(\rho_0\ g\beta_T\Delta_T H), \quad T^* = (T-T_0)/\Delta_T \end{aligned} \tag{2}$$

Then, Eq. (1) can be rewritten in dimensionless form as

$$\nabla\cdot\mathbf{u}^* = 0 \tag{3a}$$

$$\frac{\partial \mathbf{u}^*}{\partial t^*} + \mathbf{u}^*\cdot\nabla\mathbf{u}^* = -\nabla P^* + \sqrt{\frac{\mathrm{Pr}}{Ra}}\nabla^2\mathbf{u}^* + T^*\tilde{\mathbf{z}} \tag{3b}$$

$$\frac{\partial T^*}{\partial t^*} + \mathbf{u}^*\cdot\nabla T^* = \sqrt{\frac{1}{\mathrm{PrRa}}}\nabla^2 T^* \tag{3c}$$

Here, H is the cell height and it is chosen as the characteristic length. $t_f = \sqrt{H/(\beta_T g\Delta_T)}$ is the free-fall time and it is chosen as the characteristic time. Δ_T is the temperature difference between heating and cooling walls. The two dimensionless parameters are the Ra and the Pr, which are defined as

$$Ra = \frac{g\beta_T\Delta_T H^3}{\nu\alpha_T}, \quad Pr = \frac{\nu}{\alpha_T} \tag{4}$$

2.2. The LB model for thermal convection

We adopt the double distribution function (DDF)-based LB model to simulate thermal convective flows with the Boussinesq approximation [33–36]. Specifically, we chose a D2Q9 discrete lattice in two-dimension (2D) or a D3Q19 discrete lattice in three-dimension (3D) for the Navier–Stokes equations to simulate fluid flows, and a D2Q5 discrete lattice in 2D or a D3Q7 discrete lattice in 3D for the energy equation to simulate heat transfer [30,37], as illustrated in Fig. 1. Here, the D2Q5 or D3Q7 model was chosen for the convection-diffusion equation to pursue computational efficiency.

To enhance the numerical stability, the multi-relaxation-time (MRT) collision operator is adopted in the evolution equations of both density and temperature distribution functions. The evolution equation of the density distribution function is written as

$$f_i(\mathbf{x}+\mathbf{e}_i\delta_t, t+\delta_t) - f_i(\mathbf{x},t) = -\left(\mathbf{M}^{-1}\mathbf{S}\right)_{ij}\left[\mathbf{m}_j(\mathbf{x},t) - \mathbf{m}_j^{(\mathrm{eq})}(\mathbf{x},t)\right] + \delta_t F_i' \tag{5}$$

where f_i is the density distribution function and $i = 0, 1, \cdots, q-1$. $\mathbf{x}$ is the fluid plarcel position, t is the time, δ_t is the time step. $\mathbf{e}_i$ is the discrete velocity along the ith direction. For D2Q9 discrete lattice, $q = 9$; for D3Q19 discrete lattice, $q = 19$. The forcing term F_i' on the right-hand side of Eq. (5) is given by $\mathbf{F}' = \mathbf{M}^{-1}\left(\mathbf{I} - \frac{\mathbf{S}}{2}\right)\mathbf{M}\tilde{\mathbf{F}}$, and the term $\mathbf{M}\tilde{\mathbf{F}}$ is given as [38–40]

$$\mathbf{M}\tilde{\mathbf{F}}_{D2Q9} = [0, 6\mathbf{u}\cdot\mathbf{F}, -6\mathbf{u}\cdot\mathbf{F}, F_x, -F_x, F_y, -F_y, 2uF_x - 2\nu F_y, uF_x + \nu F_y]^T \tag{6a}$$

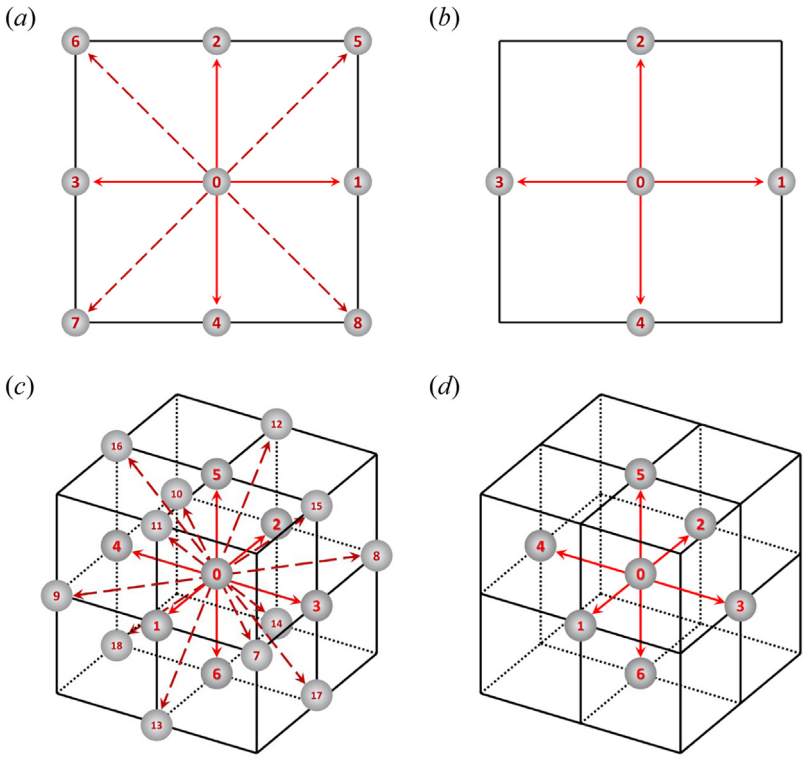

Fig. 1. Illustration of discrete velocity models: (*a*) the D2Q9 model and (*b*) the D2Q5 model for two-dimensional simulations; (*c*) the D3Q19 model and (*d*) the D3Q7 model for three-dimensional simulation.

$$\mathbf{M}\tilde{\mathbf{F}}_{D3Q19} = \begin{bmatrix} 0, 38\mathbf{u}\cdot\mathbf{F}, -11\mathbf{u}\cdot\mathbf{F}, F_x, -\frac{2}{3}F_x, F_y, -\frac{2}{3}F_y, F_z, -\frac{2}{3}F_z, \\ 4uF_x - 2vF_y - 2wF_z, -2uF_x + vF_y + wF_z, 2vF_y - 2wF_z, \\ -vF_y + wF_z, uF_y + vF_x, vF_z + wF_y, uF_z + wF_x, 0, 0, 0 \end{bmatrix}^T \tag{6b}$$

where $\mathbf{F} = \rho g\beta_T(T - T_0)\hat{\mathbf{y}}$ in 2D or $\mathbf{F} = \rho g\beta_T(T - T_0)\hat{\mathbf{z}}$ in 3D. The macroscopic density ρ and velocity $\mathbf{u}$ are obtained from $\rho = \sum_{i=0}^{q-1} f_i$, $\mathbf{u} = \left(\sum_{i=0}^{q-1} \mathbf{e}_i f_i + \frac{1}{2}\mathbf{F}\right)/\rho$. At the fluid-solid boundary, the no-slip velocity boundary condition can be realized via the half-way bounce-back scheme $f_{\bar{i}}(\mathbf{x}, t + \delta_t) = f_i^+(\mathbf{x}, t)$. Here, $f_i^+(\mathbf{x}, t)$ denotes the post-collision density distribution function; $f_{\bar{i}}(\mathbf{x}, t)$ denotes the density distribution function associated with the velocity $\mathbf{e}_{\bar{i}}$, and we have the relation $\mathbf{e}_{\bar{i}} = -\mathbf{e}_i$.

The evolution equation of the temperature distribution function is written as

$$g_i(\mathbf{x} + \mathbf{e}_i\delta_t, t + \delta_t) - g_i(\mathbf{x}, t) = -\left(\mathbf{N}^{-1}\mathbf{Q}\right)_{ij}\left[\mathbf{n}_j(\mathbf{x}, t) - \mathbf{n}_j^{(\mathrm{eq})}(\mathbf{x}, t)\right] \tag{7}$$

where g_i is the temperature distribution function and $i = 0, 1, \cdots, q-1$. For D2Q5 discrete lattice, $q = 5$; for D3Q7 discrete lattice, $q = 7$. The macroscopic temperature T is obtained from $T = \sum_{i=0}^{q-1} g_i$. At the fluid-solid boundary, the Dirichlet temperature boundary condition can be realized via the half-way anti-bounce-back scheme $g_{\bar{i}}(\mathbf{x}, t + \delta_t) = -g_i^+(\mathbf{x}, t) + \omega T_w$, where T_w is the wall temperature, $\omega = (4 + a_T)/10$ for D2Q5 and $\omega = (6 + a_T)/21$ for D3Q7, a_T is a constant related to thermal diffusivity [41]; the Neumann adiabatic boundary condition can be realized via the half-way bounce-back scheme $g_{\bar{i}}(\mathbf{x}, t + \delta_t) = g_i^+(\mathbf{x}, t)$. Here, $g_i^+(\mathbf{x}, t)$ denotes the post-collision temperature distribution function; $g_{\bar{i}}(\mathbf{x}, t)$ denotes the temperature distribution function associated with the velocity $\mathbf{e}_{\bar{i}}$. More numerical details of the thermal LB method can be found in our previous work [30,37].

2.3. Simulation settings and parallel performance characterization

We consider a 2D square cell and a 3D cubic cell. The left and right walls of the cell are kept at constant hot and cold temperatures, respectively; while the other two (or four) walls of the 2D cell (or the 3D cell) are adiabatic; all walls impose no-slip velocity boundary conditions. We fixed the Prandtl number as $Pr = 0.71$. For the 2D thermal convection, the Rayleigh number is fixed as $Ra = 10^8$; while for the 3D thermal convection, the Rayleigh number is fixed as $Ra = 10^7$. Previously, we obtained a steady flow pattern in these two cases [30,37]; while at higher Ra, the flow transits to an unsteady state, and a Hopf bifurcation occurs. We verified the multi-GPU implementation can give correct simulation results consistent with our previous results [30,37] (see Fig. 2 for the temperature field in the convection cell); however, for the sake of clarity, we do not repeat to provide the tabulated flow quantities and heat transfer properties. In the following, we focus on the parallel performance of the multi-GPU simulation.

In the strong scaling test, we measure the running time as a function of GPU numbers for fixed total problem size, which represents the ability to solve a problem faster using more resources; while in the weak scaling test, we measure the running time as a function of GPU numbers for fixed problem size per GPU (i.e., the total problem size increases), which represents the ability to solve larger problems with larger resources. We adopt two metrics to characterize the parallel performance of the LB simulation, one is Million Lattice Updates Per Second (MLUPS) and the other is parallel efficiency. The MLUPS is defined as [42]

$$\mathrm{MLUPS} = \frac{\text{mesh size} \times \text{iteration steps}}{\text{running time} \times 10^6} \tag{8}$$

Meanwhile, we have 1 GLUPS = 1000 MLUPS, where GLUPS stands for billion lattice updates per second. In the strong scaling test, the

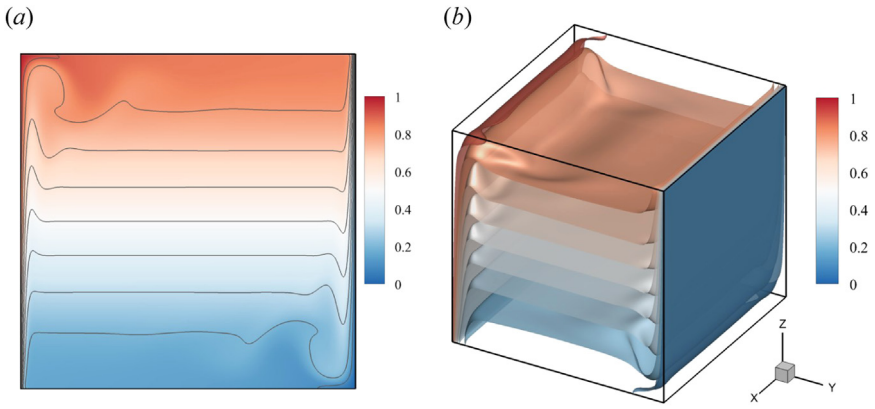

Fig. 2. The temperature field in the convection cell: (*a*) the 2D simulation and (*b*) the 3D simulation.

parallel efficiency η is defined as

$$\eta = \frac{T_1}{T_n \cdot n} \tag{9}$$

Here, T_1 denotes the running time using a single GPU and T_n denotes the running time using n GPUs. In the weak scaling test, the parallel efficiency η is simply calculated as $\eta = T_1/T_n$. We conducted experiments on a four-node GPU cluster, each node is equipped with four NVIDIA A100 GPUs powered by the NVIDIA Ampere Architecture. The network interconnects use 100 Gigabits per second (Gbps) Remote direct memory access over Converged Ethernet (RoCE). The inter-GPU-GPU communication within a node goes over the PCI-e. To automatically utilize the GPUDirect acceleration technologies, we adopt a CUDA-aware MPI implemented in OpenMPI. With GPUDirect technology, including Peer to Peer (P2P) and Remote Direct Memory Access (RDMA), the buffers can be directly sent from a GPU memory to another GPU memory or the network without touching the host memory [43].

We measure the running time and then calculate the MLUPS and parallel efficiency. In Section 3, we describe optimization strategies for multi-GPU simulation and evaluate the parallel performance via the strong scaling test; then, in Section 4, we further discuss the parallel performance via the weak scaling test. In the strong scaling test, we fix a grid number of 8193×8193 in the 2D simulation and a grid number of $385 \times 385 \times 385$ in the 3D simulation. Here, we use an odd number of grid points in each dimension for two reasons: first, it reduces the oscillations due to spurious conserved quantities [44]; secondly, for a grid number of N in each dimension, it allows the $[(N+1)/2]$th point to be precisely located at the center line (plane) based on the halfway (anti-)bounce-back boundary scheme, which is convenient for post-analysis of flow and heat transfer quantities. The iteration steps are fixed as 12,000 and 6000, respectively, for the 2D and 3D simulations. Preliminary tests showed that the corresponding running time was around 30 seconds with 16 GPUs (i.e., the largest number of GPUs in the tests), which ensures the measurement of computing performance enters a steady state. Detailed settings for the weak scaling test are described in Section 5. All the simulations adopt double-precision floating-point arithmetic, which ensures the simulation accuracy.

3. Implementation and optimization of the hybrid OpenACC and MPI approach

3.1. Naive implementation of hybrid OpenACC and MPI approach

To utilize the computing power of GPU clusters, we use the hybrid OpenACC and MPI approach, in which OpenACC accelerates the computation on a single GPU and MPI synchronizes the information between multiple GPUs. In Fig. 3(a), we present the flowchart of the DDF-based LB model for thermal convection problems. Here, we split the evaluation of the density distribution function (i.e., Eq. (5)) into the following two steps

$$Collision\ step: f_i^+(\mathbf{x},t) = f_i(\mathbf{x},t) - \left(\mathbf{M}^{-1}\mathbf{S}\right)_{ij}\left[\mathbf{m}_j(\mathbf{x},t) - \mathbf{m}_j^{(\mathrm{eq})}(\mathbf{x},t)\right] + \delta_t F_i' \tag{10a}$$

$$Streaming\ step: f_i(\mathbf{x}+\mathbf{e}_i\delta_t, t+\delta_t) = f_i^+(\mathbf{x},t) \tag{10b}$$

Similarly, we split the evaluation of the temperature distribution function (i.e., Eq. (7)) into two steps as

$$Collision\ step: g_i^+(\mathbf{x},t) = g_i(\mathbf{x},t) - \left(\mathbf{N}^{-1}\mathbf{Q}\right)_{ij}\left[\mathbf{n}_j(\mathbf{x},t) - \mathbf{n}_j^{(\mathrm{eq})}(\mathbf{x},t)\right] \tag{11a}$$

$$Streaming\ step: g_i(\mathbf{x}+\mathbf{e}_i\delta_t, t+\delta_t) = g_i^+(\mathbf{x},t) \tag{11b}$$

In the collision step, we can use the symbol $\mathbf{\Omega}_i(\mathbf{x},t)$ to denote the collision operator, which is the second term on the right-hand side of the equation. The collision step is also known as the relaxation step and it completely updates local information; the streaming step is also known as the propagation step and it transfers data information to its neighboring. Because GPUs adopt single instruction multiple threads (SIMT) execution model, we optimized the data layout for the distribution function and stored the information in the structure of array (SoA) to meet the requirement of coalescing memory access, such that neighboring threads access neighboring data. For example, the density distribution function $f_i(\mathbf{x},t)$ is stored with index $(x + N_x \times y + N_x \times N_y \times i)$ or $(x + N_x \times y + N_x \times N_y \times z + N_x \times N_y \times N_z \times i)$, respectively, for the 2D or 3D case. Here, N_x, N_y, and N_z denote the grid number in the x, y, and z directions, respectively. Using the column-major order programming language (such as Fortran), the corresponding index formula implementation is $f(x,y,i)$ or $f(x,y,z,i)$ [30].

In a naive implementation of the hybrid OpenACC and MPI approach, we adopt a mono-dimensional partitioning of the computational domain. Take the column-major order programming language (such as Fortran) as an example, we decompose the domain along the y-direction (or the z-direction) in the 2D (or the 3D) domain, then the interface between the subdomains is along the x-direction line (or the $x-y$ plane), which favors the continuous transfer data in memory space. We add ghost layers outside the boundary of each subdomain to receive data from the adjacent subdomains, as illustrated in Fig. 3(b). To reduce the amount of data transfer, in the streaming step, we do not transfer the full array of the distribution function $f_i^+(\mathbf{x})$ or $g_i^+(\mathbf{x})$, but only components of these arrays corresponding to specific discrete velocity directions. For example, using the D2Q9 discrete lattice for the

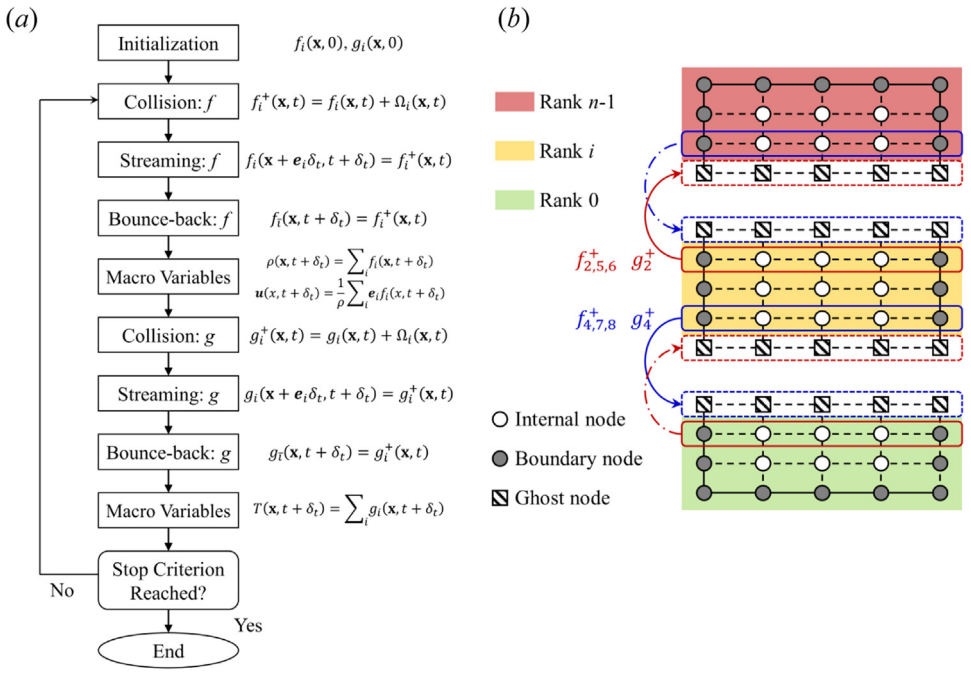

Fig. 3. (*a*) Flowchart of double distribution function (DDF)-based lattice Boltzmann (LB) model for thermal convection simulation; (*b*) mono-dimensional partitioning of the 2D computation domain and the associated data exchange.

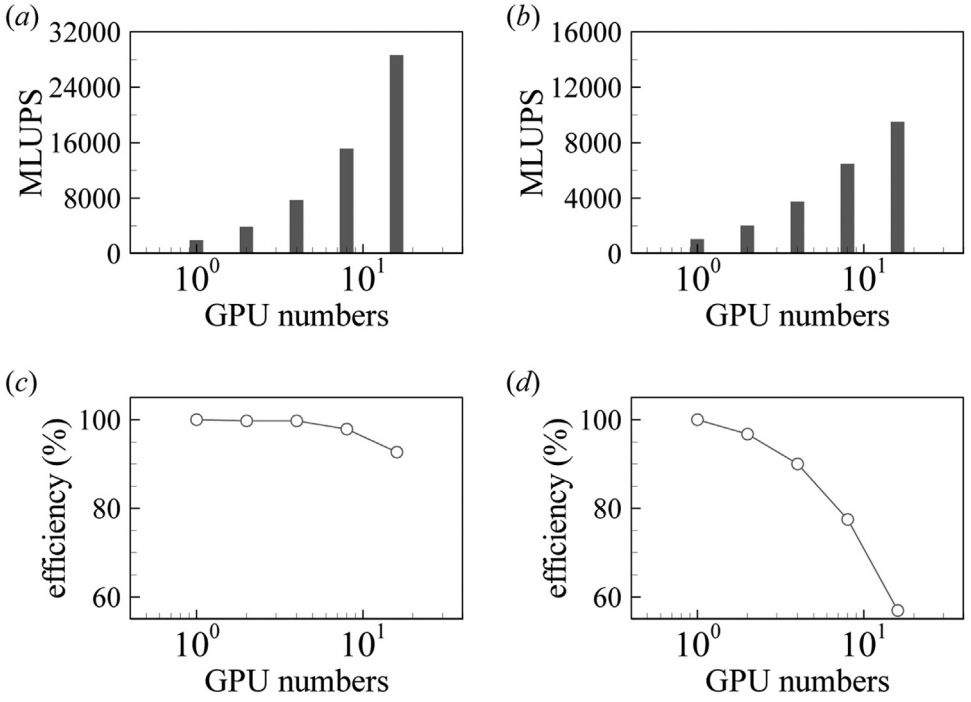

Fig. 4. Performanc of (*a, c*) the 2D simulation and (*b, d*) the 3D simulation, in terms of (*a, b*) the MLUPS and (*c, d*) the parallel efficiency.

density distribution function and D2Q5 discrete lattice for the temperature distribution function, the current GPU [the corresponding subdomain is marked by orange color in Fig. 3(b)] only sends values of $f_{2,5,6}^+(\mathbf{x})$ and $g_2^+(\mathbf{x})$ [these are distribution functions belong to the top boundary of the subdomain, see the red rectangle with the solid line in Fig. 3(b)] to the top neighbor GPU, while it only sends values of $f_{4,7,8}^+(\mathbf{x})$ and $g_4^+(\mathbf{x})$ [these are distribution functions belong to the bottom boundary of the subdomain, see the blue rectangle with the solid line in Fig. 3(b)] to the bottom neighbor GPU.

Fig. 4 shows the parallel performance in terms of MULPS and parallel efficiency for the 2D simulation and the 3D simulation, respectively. We can see that using 1 GPU, the 2D thermal LB simulation achieves 1931.2 MLUPS, and the 3D thermal LB simulation achieves 1042.4 MLUPS. We note in the D2Q9 + D2Q5 thermal LB model, each grid node accesses 80 variables within an iteration step and these variables occupy 8 bytes in double-precision; while in the D3Q19 + D3Q7 thermal LB model, each grid node accesses 143 variables within an iteration step. Because the NVIDIA A100 has a memory bandwidth of 1555 GB/s, the

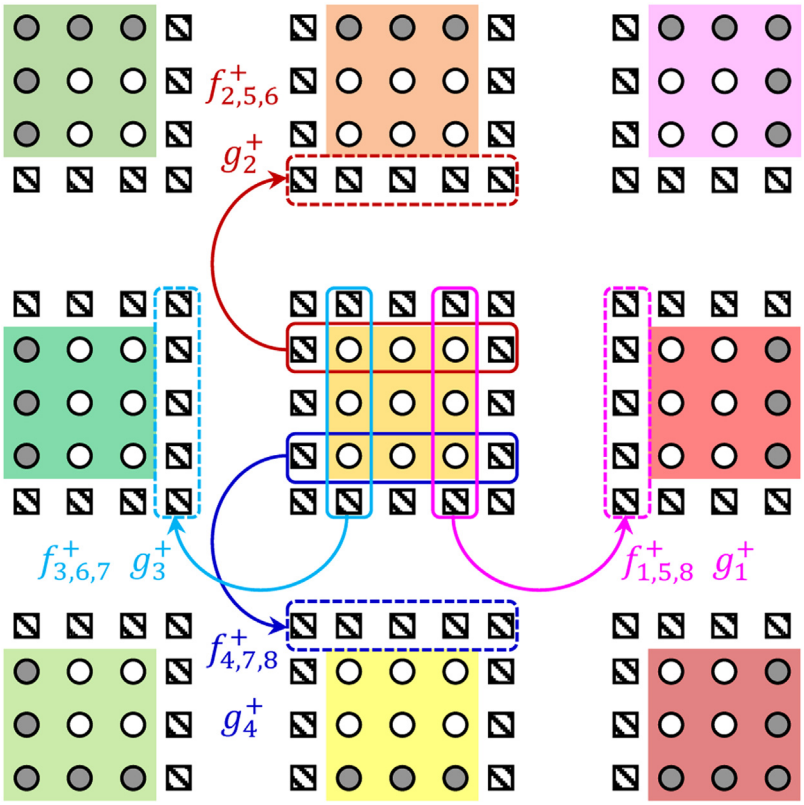

Fig. 5. Illustration of block partitioning and data exchange for the 2D domain.

thermal LB simulation has a theoretical maximum performance of $1555 \times 1000^3/(80 \times 8 \times 10^6) = 2429.7$ MLUPS for the 2D simulation, and $1555 \times 1000^3/(143 \times 8 \times 10^6) = 1359.3$ MLUPS for the 3D simulation. In other words, we have reached 79.5% and 76.7% of the theoretical maximum performance, similar to that of the SunwayLB [14]. With the increase of GPU numbers, the MULPS generally increases; however, the parallel efficiency degrades when more GPUs are used, particularly for the 3D case in which the parallel efficiency is less than 57% using 16 GPUs, indicating the parallel code is not scalable. Two reasons may be responsible for the poor parallel performance. First, in the strong scaling measurement, the computational load on each GPU decreases with the increase of GPU numbers; however, the communication cost per GPU almost remains the same. Adopting the mono-dimensional partitioning, the amount of data that needs to be transferred is $2 \times N_x$ and $2 \times N_x \times N_y$, respectively, for the 2D and the 3D simulation. As a result, the ratio between communication time and the total time increases with the increase of GPU numbers, restricting the scalability of the code. Secondly, if the grid number of the whole computation domain is not large enough, the computational load on each GPU is not sufficient to fully occupy the resources of that GPU, and the overhead to launch kernel matters. To further boost the parallel performance using multi-GPUs, we describe some optimization strategies in the following subsections.

3.2. Block partitioning of the computational domain

In mono-dimensional partitioning, the computational domain is decomposed into several slices and each slice is allocated to a single GPU. An alternative strategy for domain decomposition is block partitioning, which is to decompose the domain in more than one dimension [45,46]. Fig. 5 illustrates the block partitioning and data exchange for the 2D domain. We decompose the domain both along the x and the y directions, and we minimize the differences in the subdomain size along each dimension. Using the column-major order programming language (such as Fortran), the data is stored continuously along the x-direction in the memory space, we preferably decompose the domain along the y-direction (in 2D) or the z-direction (in 3D). The partition details for the domain in 2D and 3D are provided in Table 1. After that, we add ghost layers to each subdomain to store the data received from adjacent subdomains. Using Fortran, we need to wrap the left and right boundary nodes along the y-direction into a contiguous cache space before sending them to its left and right neighboring GPUs (see the nodes in the light blue and purple rectangles with solid lines in Fig. 5). Here we also included ghost nodes outside the subdomain boundaries when sending the message, such that data for the corner points can be implicitly synchronized and transferred to the diagonally neighboring GPUs. With CUDA-aware MPI libraries implemented in OpenMPI, we discourage the use of MPI-derived datatypes for data communication, even though both contiguous and non-contiguous derived datatypes are supported. The non-contiguous datatypes currently have a high overhead because of the many calls to copy all the pieces of the buffer into the intermediate buffer. Previously, Calore et al. [47] overcame this issue by developing a custom communication library that uses persistent send and receives buffers, allocated once on the GPUs at program initialization. Here, we recommend an alternative solu-

Table 1
Partition details of the domain in 2D and 3D.

GPU numbers	Partition type in 2D	partition type in 3D
1	-	-
2	1×2	$1 \times 1 \times 2$
4	2×2	$1 \times 2 \times 2$
8	2×4	$2 \times 2 \times 2$
16	4×4	$2 \times 2 \times 4$

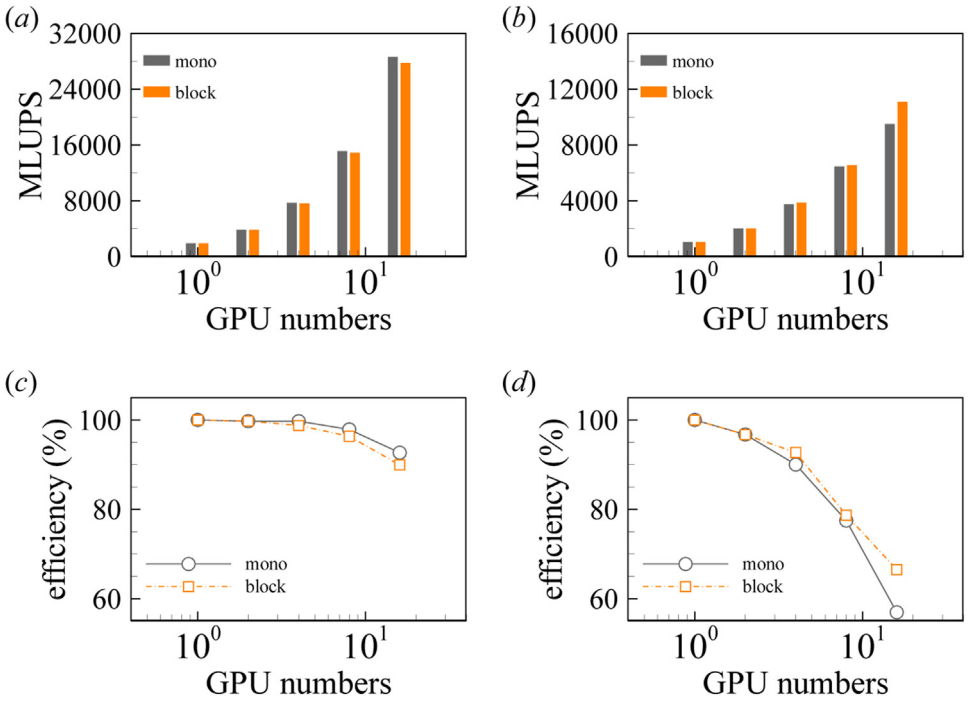

Fig. 6. Performanc comparisons between mono-dimensional and block partitioning of the computational domain for (*a, c*) the 2D simulation and (*b, d*) the 3D simulation, in terms of (*a, b*) the MLUPS and (*c, d*) the parallel efficiency.

tion that simply put the ghost nodes into a buffer of contiguous memory locations.

Fig. 6 compares the performance between mono-dimensional partitioning and block partitioning of the computational domain. We can see that adopting block partitioning can improve the MLUPS and parallel efficiency for the 3D simulations, however, it slightly degrades the parallel performance for the 2D simulations. In the following, we provide a theoretical analysis of the advantages and disadvantages of adopting block partitioning. For the 2D computational domain with size $N_x \times N_y$, we assume a square domain of $N_x = N_y = N$ for simplicity. Adopting the mono-dimensional partitioning, each GPU sends $2N$ boundary nodes to its neighboring GPUs, and it launches two kernels for data communications on the two boundaries; while adopting the block partitioning, each GPU sends $4(N/\sqrt{a}+2)$ boundary nodes to its neighboring, and it launches four kernels for data commutations on the four boundaries. Here, for simplicity, we discuss the case when the GPU number a meets the requirement that $\sqrt{a}$ is an integer. We denote that the time consumption for sending a single boundary node is k, and the time consumption to launch a kernel for data communication is h. Thus, for mono-dimensional and block partitioning, the total time consumption for message passing, which includes the time for sending data and the time to launch a kernel, on a single GPU is $t_1 = 2kN + 2h$ and $t_2 = 4(N/\sqrt{a}+2)k + 4h$, respectively. The above analysis indicates that the block partitioning would be more efficient when $t_1 > t_2$, i.e., $2kN - 4Nk/\sqrt{a} > 2h + 8k$; in other words, we prefer to use block partitioning when both the domain size N and the GPU number a are large.

3.3. Overlapping communications with computations

Minimizing the communication overhead is crucial to improve the parallel performance of the simulations based on multi-GPUs, we further hide communication overhead behind the kernel runtime by overlapping the communications with computations. The basic idea is to simultaneously execute kernels on GPUs for intense computation and data communications among GPUs, because GPUs cannot control data transfer and only CPUs can manage the data communication. On devices with distinct hosts and device memory, we can create asynchronous work queues to deal with the cost of PCIe data transfers. Practically, we can adopt the *!$acc async* handle in OpenACC. As illustrated in Fig. 7(a), we first update the boundary nodes and put them into buffers of contiguous memory. We then create asynchronous work queues to perform intense computation for updating inner nodes, while synchronizing data of boundary nodes among various GPUs using MPI. Thus, the latency of communication can be hidden behind the computations. The next task will not begin execution until all actions on the async queues are complete, which can be realized using the *!$acc wait* handle in OpenACC. In Fig. 7(b), we give an example of the thermal LB model for overlapping communications with computations. Because the evolution of density distribution function f_i and temperature distribution function g_i are independent of each other within an iteration, we can update f_i and g_i alternatively to hide the communication overhead. Meanwhile, the amount of data computation for f_i is larger than that for g_i, since there are 5 components of f_i for the D3Q19 discrete lattice and 1 component of g_i for the D3Q7 discrete lattice on a surface of the subdomain, we then divide the communication of f_i into three sets: f_x representing the density distribution at the back and front surfaces, f_y representing the density distribution function at the left and right surfaces, and f_z representing the density distribution function at the bottom and top surfaces. As illustrated in Fig. 7(b), we first relax g_i at the inner nodes while communicating f_z. After that, we communicate g_i at the boundary nodes while relaxing f_i at the inner nodes. Because the relaxation of f_i is the most time-consuming subroutine, as evident by the results obtained via NVIDIA Nsight Systems and represented by the box size in the illustration drawing, we can also hide the communication of f_y while relaxing f_i at the inner nodes. For the communication of f_x, it can be hidden by the propagation step of g_i and the step to calculate macroscopic variables of T. Now that the data communication for f_i and g_i at the boundary nodes

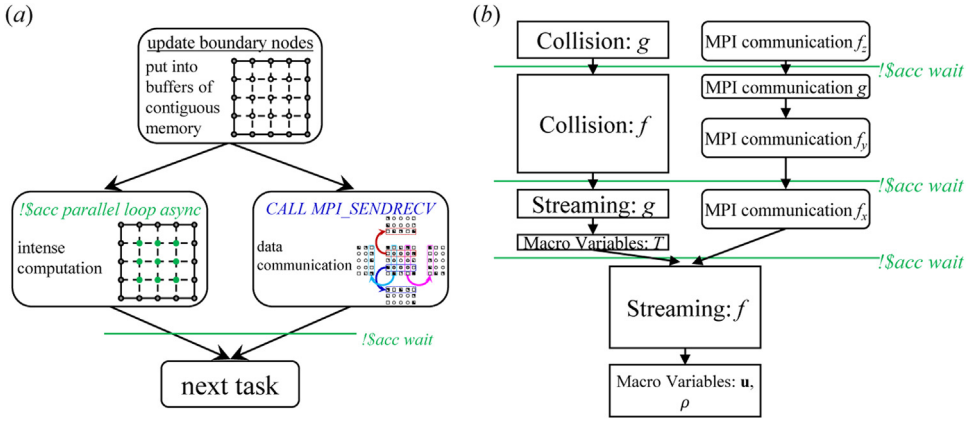

Fig. 7. Schematic illustration of (*a*) the basic idea to overlap communications with computations in OpenACC and (*b*) an example in the thermal LB model for overlapping communications with computations. The size of the box to represent each computation subroutine is generally proportional to its time consumption (obtained via NVIDIA Nsight Systems). The bounce-back routines updating the distribution functions at boundaries are neglected because they account for less than 1% of the total time.

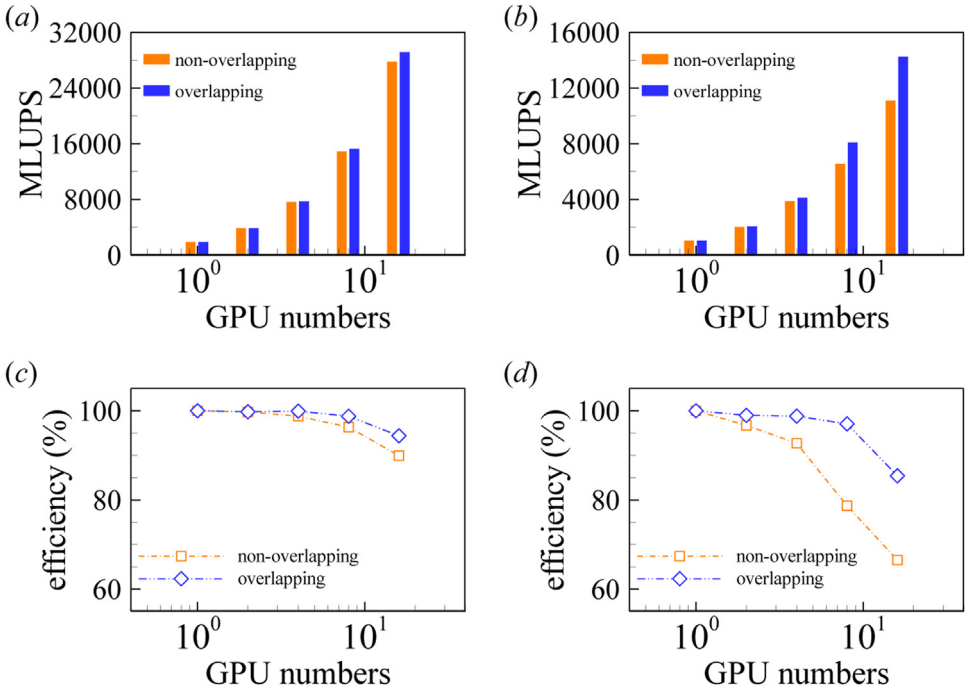

Fig. 8. Performanc comparisons between non-overlapping and overlapping communication with computation for (*a, c*) the 2D simulation and (*b, d*) the 3D simulation, in terms of (*a, b*) the MLUPS and (*c, d*) the parallel efficiency.

are completed within an iteration step, we do not have to overlap communications for the remaining computation subroutines of propagation f_i and calculation macroscopic variables of $\mathbf{u}$ and ρ.

Fig. 8 compares the performance of non-overlapping and overlapping communication with computation. We can see that the MLUPS and parallel efficiency improved for all the cases if the communications and computations overlapped, and the advantage of using the overlapping mode is more obvious with the increase of GPU numbers. For the 3D simulations, the performance improvement is greater than that in the 2D simulations, because more time for data transfer between GPU communication is hidden. Excitingly, for the 3D simulation, the parallel efficiency improves from 78.7% to 97.1% when using 8 GPUs, and the MLUPS increases by 1.28X when using 16 GPUs. Previously, using the overlapping mode implemented in a hybrid CUDA and MPI approach, Xian and Takayuki [48] reported the MLUPS increased by 1.24X with 16 GPUs, Hong et al. [49] reported the MLUPS increased by 1.38X with 6 GPUs, suggesting a similar amount of increase in the parallel performance even though we adopt the high level and directive-based OpenACC standard.

3.4. Concurrent computation on a GPU

In addition to data parallelism, exploiting task parallelism can further utilize the GPU hardware resources, particularly when using a large number of GPUs, the computational tasks may not fully occupy the resources of the GPU, and we can concurrently execute two independent tasks on a single GPU, as illustrated in Fig. 9(a). Practically, we can adopt the *!$acc async (n)* handle, which launches work asynchronously in queue *n*. All operations in the same queue will execute in order, while operations in different queues may execute in any order. In Fig. 9(b), we give an example of the thermal LB model for concurrent computation. Because the evolution of density distribution function f_i and temperature distribution function g_i are independent of each other within an iteration, we can also update f_i and g_i in different queues. It should be

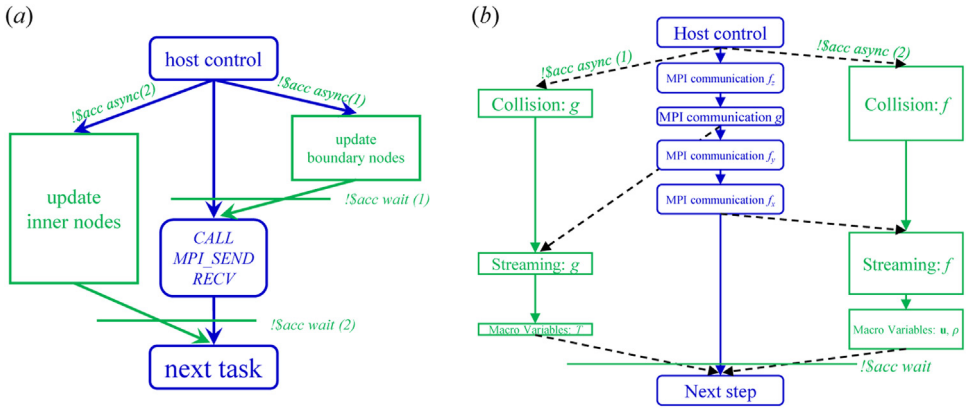

Fig. 9. Schematic illustration of (*a*) the basic idea of concurrently executing two independent tasks in OpenACC and (*b*) an example in the thermal LB model for concurrent computation.

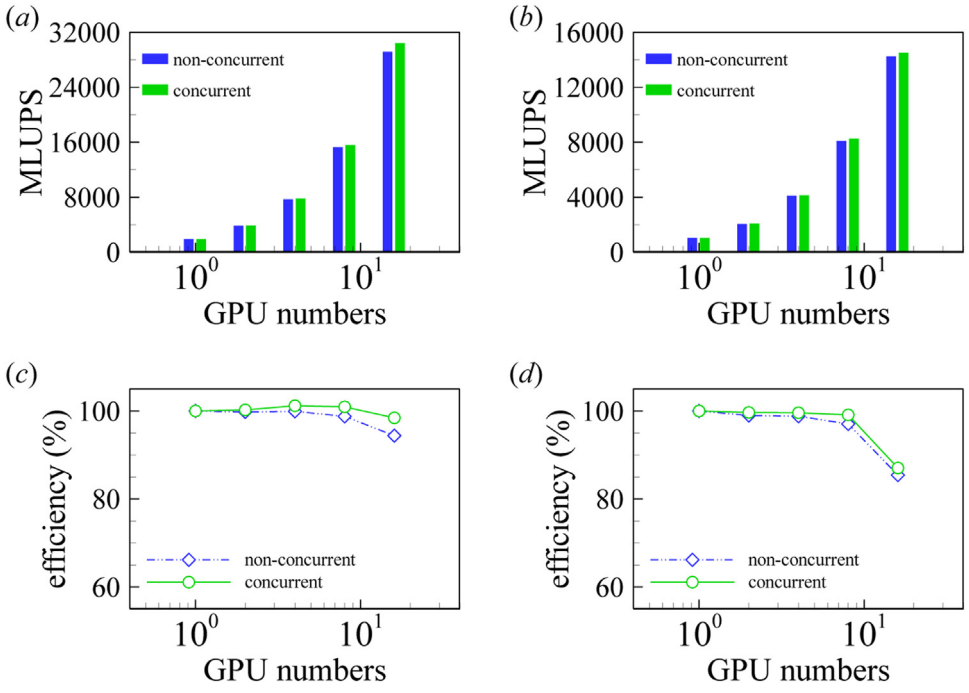

Fig. 10. Performanc comparisons between non-concurrent and concurrent computation for (*a, c*) the 2D simulation and (*b, d*) the 3D simulation, in terms of (*a, b*) the MLUPS and (*c, d*) the parallel efficiency.

noted that the data communication at the boundary nodes should be performed before the propagation step. Here, following the discussion in Section 3.3, we divide the communication of f_i into three sets, and we overlap the communication of f_i and g_i [see the blue box in Fig. 9(b)] with computation [see the green box in Fig. 9(b)].

Fig. 10 compares the performance of non-concurrent and concurrent computation. We can see the MLUPS and the parallel efficiency slightly improves. Using 16 GPUs, the 2D simulation achieved 30.42 GLUPS, and the 3D simulation achieved 14.52 GLUPS. An interesting finding is that for the 2D simulation, the parallel efficiency may even be greater than 100%, which may be due to the efficient use of the on-chip memory on the GPU, highlighting the advantage of exploiting task parallelism. Due to the high ability of GPU for computation, the GPU performance for the 3D simulation can be further boosted with the increasing of computational load.

4. Weak scaling test

In the weak scaling test, we use a sub-domain size of 8192×8192 and $384 \times 384 \times 384$ in the 2D and the 3D simulation, respectively; the iteration steps are fixed as 1000 (in 2D) and 500 (in 3D). Each dimension of the domain size increases similarly to that of increasing in GPU numbers (see Table 1 for the partition details of the domain). For simplicity, we only provide the performance of the code after adopting all the optimization strategies described in Section 3. As shown in Fig. 11, using 1 GPU, we can achieve 1932.9 MLUPS and 1045.3 MLUPS in the 2D and 3D simulation (less than 0.3% deviation from that in the strong scaling test due to different runs), respectively. With the increase of GPU numbers, the MLUPS almost increases linearly up to 16 GPUs, and the parallel efficiency remains above 99%. These results demonstrate that the optimized thermal LB code has excellent weak scalability.

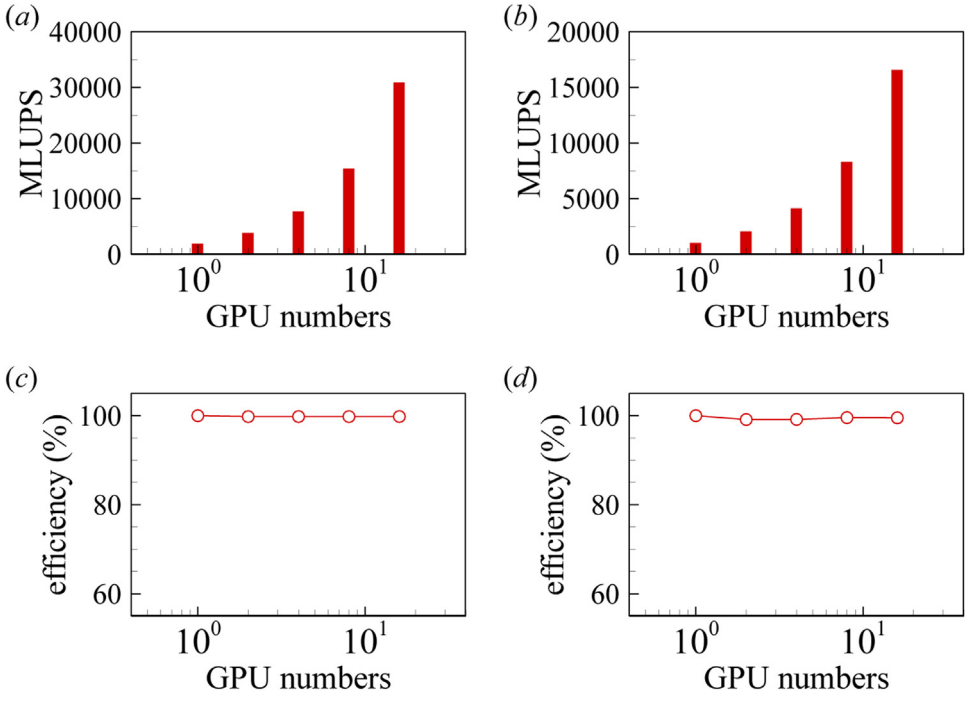

Fig. 11. Weak scaling test: performance of (*a, c*) the 2D simulation and (*b, d*) the 3D simulation, in terms of (*a, b*) the MLUPS and (*c, d*) the parallel efficiency.

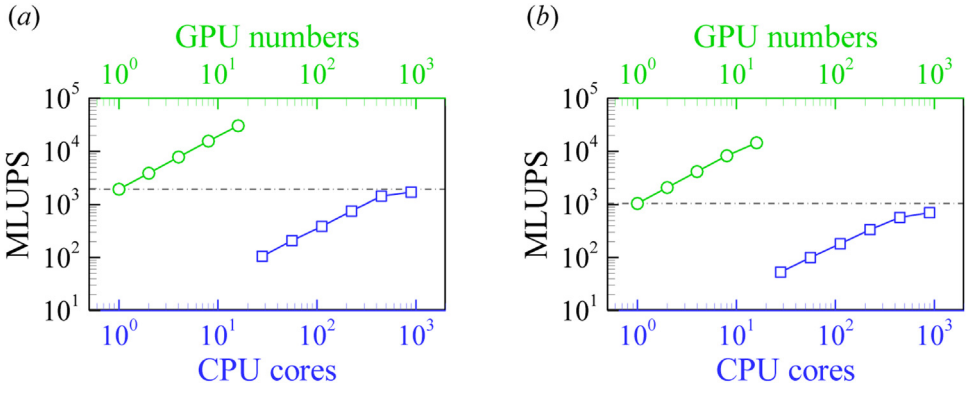

Fig. 12. Comparison of the parallel performance on CPUs versus GPUs for (*a*) the 2D simulation and (*b*) the 3D simulation in terms of the MLUPS. The gray-dashed lines denote the MLUPS on a single GPU. The total number of CPU cores is $N_1 \times N_2$, where N_1 is the number of MPI processes and N_2 is the number of OpenMP threads per MPI process. Here, we fix N_2 as 14 and N_1 increases from 2 to 64.

5. Conclusions

In this work, we have adopted a hybrid OpenACC and MPI approach for accelerated thermal LB simulation on multi-GPUs. The OpenACC accelerates computation on a single GPU, and the MPI synchronizes the information between multiple GPUs. We adopt the double distribution function-based thermal LB model, namely, D2Q9 + D2Q5 in the 2D simulation, and D3Q19 + D3Q7 in the 3D simulation. With a single NVIDIA A100 GPU, the 2D simulation achieved 1.93 GLUPS with a grid number of 8193^2 and the 3D thermal LB simulation achieves 1.04 GLUPS with a grid number of 385^3, which is more than 76% of the theoretical maximum performance. In a naive implementation to extend to multi-GPUs, we used mono-direction partitioning of the computation domain, however, the code was not scalable to more than 8 GPUs in the 3D simulation. To further boost the parallel performance, we adopted three optimization strategies: block partitioning, overlapping communications with computations, and concurrent computation.

With block partitioning, the domain is decomposed in more than one dimension, and it decreases the amount of data that needs to be transferred. By overlapping the communications with computations, communication overhead is hidden behind the kernel runtime. Using concurrent computation, task parallelism can be exploited to better utilize the GPU hardware resources. After adopting these optimization strategies, we demonstrate that the parallel performance can be significantly improved. In the strong scaling test, using 16 GPUs, the 2D simulation achieved 30.42 GLUPS and the 3D simulation achieved 14.52 GLUPS. In the weak scaling test, the parallel efficiency remains above 99% up to 16 GPUs. It should be noted that all performance measurements are based on double-precision floating-point arithmetic, which ensures simulation accuracy. Our results demonstrated that, with improved data and task management, the hybrid OpenACC and MPI technique is promising for thermal LB simulation on multi-GPUs.

Declaration of Competing Interest

We confirm that the manuscript has been read and approved by all named authors and that there are no other persons who satisfied the criteria for authorship but are not listed. We further con-

firm that the order of authors listed in the manuscript has been approved by all of us.

CRediT authorship contribution statement

Ao Xu: Conceptualization, Software, Formal analysis, Resources, Writing – original draft, Writing – review & editing, Supervision, Project administration, Funding acquisition. **Bo-Tao Li:** Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Visualization.

Data Availability

Data will be made available on request.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (NSFC) through Grant Nos. 11902268 and 12272311, the Open Fund of Key Laboratory of Icing and Anti/De-icing (Grant No. IADL20200301). and the National Key Project via No. GJXM92579, The authors acknowledge the Beijing Beilong Super Cloud Computing Co., Ltd for providing HPC resources that have contributed to the research results reported within this paper (URL: http://www.blsc.cn/).

Appendix A. Comparison of the parallel performance on CPUs versus GPUs

In the appendix, we summarize the MLUPS of the GPU code adopting the optimization strategies discussed in Section 3. As a comparison, we provide the MLUPS of a CPU code, which is developed with a hybrid MPI and OpenMP approach [50]. The CPU code is based on a hierarchical two-level parallelization, where the first-level parallelization applies MPI domain decomposition to the simulation domain, and the second-level parallelization uses OpenMP parallel regions for loops within a subdomain. Such a hybrid MPI + OpenMP approach can reduce the memory usage and overhead associated with MPI calls. For experiments on the CPU cluster, each node is equipped with two Intel Xeon 6258R CPUs (i.e., 56 cores within a node), we assign 4 MPI processes on each node with 14 OpenMP threads per MPI process. This choice is a compromise between two factors: first, more OpenMP threads reduces the corresponding number of MPI processes, which leads to better communication performances; secondly, more OpenMP threads increases the overhead associated with OpenMP constructs and remote memory accesses across sockets. For all the cases investigated, our test results showed that on each node, the combination of 4 MPI processes $\times$ 14 OpenMP threads works slightly better than that of 7 MPI processes $\times$ 8 OpenMP threads. We can see from Fig. 12 that in the strong scaling test, both our CPU code and GPU code scale well.

References

[1] R.P. Feynman, Plenty of room at the bottom, APS Annual Meeting, 1959.
[2] G.E. Moore, et al., Progress in digital integrated electronics, in: Electron Devices Meeting, Vol. 21, 1975, pp. 11–13. Washington, DC
[3] J. Shalf, The future of computing beyond Moore's law, Philos. Trans. R. Soc. A-Math. Phys. Eng. 378 (2166) (2020) 20190061.
[4] C.E. Leiserson, N.C. Thompson, J.S. Emer, B.C. Kuszmaul, B.W. Lampson, D. Sanchez, T.B. Schardl, There's plenty of room at the top: what will drive computer performance after Moore's law? Science 368 (6495) (2020). Eaam9744
[5] S. Chen, G.D. Doolen, Lattice Boltzmann method for fluid flows, Annu. Rev. Fluid Mech. 30 (1) (1998) 329–364.
[6] C.K. Aidun, J.R. Clausen, Lattice-Boltzmann method for complex flows, Annu. Rev. Fluid Mech. 42 (2010) 439–472.
[7] P. Cheng, X. Quan, S. Gong, X. Liu, L. Yang, Recent analytical and numerical studies on phase-change heat transfer, in: Advances in Heat Transfer, Vol. 46, Elsevier, 2014, pp. 187–248.
[8] Q. Li, K.H. Luo, Q. Kang, Y. He, Q. Chen, Q. Liu, Lattice Boltzmann methods for multiphase flow and phase-change heat transfer, Prog. Energy Combust. Sci. 52 (2016) 62–105.
[9] M. Maxey, Simulation methods for particulate flows and concentrated suspensions, Annu. Rev. Fluid Mech. 49 (2017) 171–193.
[10] S. Tao, L. Wang, Q. He, J. Chen, J. Luo, Lattice Boltzmann simulation of complex thermal flows via a simplified immersed boundary method, J. Comput. Sci. (2022) 101878.
[11] Q. Xiong, B. Li, G. Zhou, X. Fang, J. Xu, J. Wang, X. He, X. Wang, L. Wang, W. Ge, et al., Large-scale DNS of gas–solid flows on mole-8.5, Chem. Eng. Sci. 71 (2012) 422–430.
[12] C. Xu, X. Liu, K. Liu, Y. Xiong, H. Huang, A free flexible flap in channel flow, J. Fluid Mech. 941 (2022) A12.
[13] Y.-L. He, Q. Liu, Q. Li, W.Q. Tao, Lattice Boltzmann methods for single-phase and solid-liquid phase-change heat transfer in porous media: a review, Int. J. Heat Mass Transf. 129 (2019) 160–197.
[14] Z. Liu, X. Chu, X. Lv, H. Meng, S. Shi, W. Han, J. Xu, H. Fu, G. Yang, SunwayLB: Enabling extreme-scale lattice Boltzmann method based computing fluid dynamics simulations on sunway taihulight, in: 2019 IEEE International Parallel and Distributed Processing Symposium (IPDPS), IEEE, 2019, pp. 557–566.
[15] G. Falcucci, G. Amati, P. Fanelli, V.K. Krastev, G. Polverino, M. Porfiri, S. Succi, Extreme flow simulations reveal skeletal adaptations of deep-sea sponges, Nature 595 (7868) (2021) 537–541.
[16] M.J. Krause, A. Kummerländer, S.J. Avis, H. Kusumaatmaja, D. Dapelo, F. Klemens, M. Gaedtke, N. Hafen, A. Mink, R. Trunk, J.E. Marquardt, M.-L. Maier, M. Haussmann, S. Simonis, OpenLB–open source lattice Boltzmann code, Comput. Math. Appl. 81 (2021) 258–288.
[17] J. Latt, O. Malaspinas, D. Kontaxakis, A. Parmigiani, D. Lagrava, F. Brogi, M.B. Belgacem, Y. Thorimbert, S. Leclaire, S. Li, F. Marson, J. Lemus, C. Kotsalos, R. Conradin, C. Coreixas, R. Petkantchin, F. Raynaud, J. Beny, B. Chopard, Palabos: parallel lattice Boltzmann solver, Comput. Math. Appl. 81 (2021) 334–350.
[18] M. Januszewski, M. Kostur, Sailfish: a flexible multi-GPU implementation of the lattice Boltzmann method, Comput. Phys. Commun. 185 (9) (2014) 2350–2368.
[19] G. Amati, S. Succi, P. Fanelli, V.K. Krastev, G. Falcucci, Projecting LBM performance on exascale class architectures: a tentative outlook, J. Comput. Sci. 55 (2021) 101447.
[20] O. Navarro-Hinojosa, S. Ruiz-Loza, M. Alencastre-Miranda, Physically based visual simulation of the lattice Boltzmann method on the GPU: a survey, J. Supercomput. 74 (7) (2018) 3441–3467.
[21] K.E. Niemeyer, C.J. Sung, Recent progress and challenges in exploiting graphics processors in computational fluid dynamics, J. Supercomput. 67 (2) (2014) 528–564.
[22] W. Li, X. Wei, A. Kaufman, Implementing lattice Boltzmann computation on graphics hardware, Visual Comput. 19 (7) (2003) 444–456.
[23] J. Tölke, M. Krafczyk, TeraFLOP computing on a desktop PC with GPUs for 3d CFD, Int. J. Comput. Fluid Dyn. 22 (7) (2008) 443–456.
[24] N. Delbosc, J.L. Summers, A. Khan, N. Kapur, C.J. Noakes, Optimized implementation of the lattice Boltzmann method on a graphics processing unit towards real-time fluid simulation, Comput. Math. Appl. 67 (2) (2014) 462–475.
[25] C. Huang, B. Shi, N. He, Z. Chai, Implementation of multi-GPU based lattice Boltzmann method for flow through porous media, Adv. Appl. Math. Mech. 7 (1) (2015) 1–12.
[26] C. Huang, B. Shi, Z. Guo, Z. Chai, Multi-GPU based lattice Boltzmann method for hemodynamic simulation in patient-specific cerebral aneurysm, Commun. Comput. Phys. 17 (4) (2015) 960–974.
[27] E. Calore, J. Kraus, S.F. Schifano, R. Tripiccione, Accelerating lattice Boltzmann applications with openACC, in: European Conference on Parallel Processing, Springer, 2015, pp. 613–624.
[28] S. Blair, C. Albing, A. Grund, A. Jocksch, Accelerating an MPI lattice Boltzmann code using openACC, in: Proceedings of the Second Workshop on Accelerator Programming using Directives, 2015, pp. 1–9.
[29] E. Calore, A. Gabbana, J. Kraus, S.F. Schifano, R. Tripiccione, Performance and portability of accelerated lattice Boltzmann applications with openACC, Concurr. Comput.-Pract. Exp. 28 (12) (2016) 3485–3502.
[30] A. Xu, L. Shi, T. Zhao, Accelerated lattice Boltzmann simulation using GPU and openACC with data management, Int. J. Heat Mass Transf. 109 (2017) 577–588.
[31] F. Kuznik, C. Obrecht, G. Rusaouen, J.J. Roux, LBM based flow simulation using GPU computing processor, Comput. Math. Appl. 59 (7) (2010) 2380–2392.
[32] C. Obrecht, F. Kuznik, B. Tourancheau, J.J. Roux, Multi-GPU implementation of the lattice Boltzmann method, Comput. Math. Appl. 65 (2) (2013) 252–261.
[33] H. Yoshida, M. Nagaoka, Multiple-relaxation-time lattice Boltzmann model for the convection and anisotropic diffusion equation, J. Comput. Phys. 229 (20) (2010) 7774–7795.
[34] Z. Chai, T. Zhao, Lattice Boltzmann model for the convection-diffusion equation, Phys. Rev. E 87 (6) (2013) 063309.
[35] J. Wang, D. Wang, P. Lallemand, L.S. Luo, Lattice Boltzmann simulations of thermal convective flows in two dimensions, Comput. Math. Appl. 65 (2) (2013) 262–286.
[36] D. Contrino, P. Lallemand, P. Asinari, L.S. Luo, Lattice-Boltzmann simulations of the thermally driven 2D square cavity at high Rayleigh numbers, J. Comput. Phys. 275 (2014) 257–272.
[37] A. Xu, L. Shi, H.D. Xi, Lattice Boltzmann simulations of three-dimensional thermal convective flows at high rayleigh number, Int. J. Heat Mass Transf. 140 (2019) 359–370.
[38] Z. Guo, C. Zheng, B. Shi, Discrete lattice effects on the forcing term in the lattice Boltzmann method, Phys. Rev. E 65 (4) (2002) 046308.

[39] Z. Guo, C. Zheng, Analysis of lattice Boltzmann equation for microscale gas flows: relaxation times, boundary conditions and the Knudsen layer, Int. J. Comput. Fluid Dyn. 22 (7) (2008) 465–473.
[40] Z. Chai, T. Zhao, Effect of the forcing term in the multiple-relaxation-time lattice Boltzmann equation on the shear stress or the strain rate tensor, Phys. Rev. E 86 (1) (2012) 016705.
[41] L. Li, R. Mei, J.F. Klausner, Boundary conditions for thermal lattice Boltzmann equation method, J. Comput. Phys. 237 (2013) 366–395.
[42] P. Bailey, J. Myre, S.D. Walsh, D.J. Lilja, M.O. Saar, Accelerating lattice Boltzmann fluid flow simulations using graphics processors, in: 2009 International Conference on Parallel Processing, IEEE, 2009, pp. 550–557.
[43] C.-C. Ye, P.-J.-Y. Zhang, Z.-H. Wan, R. Yan, D.J. Sun, Accelerating CFD simulation with high order finite difference method on curvilinear coordinates for modern GPU clusters, Adv. Aerodynam. 4 (1) (2022) 1–32.
[44] L.-S. Luo, W. Liao, X. Chen, Y. Peng, W. Zhang, et al., Numerics of the lattice Boltzmann method: effects of collision models on the lattice Boltzmann simulations, Phys. Rev. E 83 (5) (2011) 056710.
[45] C. Schepke, N. Maillard, P.O. Navaux, Parallel lattice Boltzmann method with blocked partitioning, Int. J. Parallel Program. 37 (6) (2009) 593–611.
[46] C. Obrecht, F. Kuznik, B. Tourancheau, J.J. Roux, Scalable lattice Boltzmann solvers for CUDA GPU clusters, Parallel Comput. 39 (6–7) (2013) 259–270.
[47] E. Calore, A. Gabbana, J. Kraus, E. Pellegrini, S.F. Schifano, R. Tripiccione, Massively parallel lattice–Boltzmann codes on large GPU clusters, Parallel Comput. 58 (2016) 1–24.
[48] W. Xian, A. Takayuki, Multi-GPU performance of incompressible flow computation by lattice Boltzmann method on GPU cluster, Parallel Comput. 37 (9) (2011) 521–535.
[49] P.-Y. Hong, L.-M. Huang, L.-S. Lin, C.A. Lin, Scalable multi-relaxation-time lattice Boltzmann simulations on multi-GPU cluster, Comput. Fluids 110 (2015) 1–8.
[50] H. Jin, D. Jespersen, P. Mehrotra, R. Biswas, L. Huang, B. Chapman, High performance computing using MPI and openMP on multi-core parallel systems, Parallel Comput. 37 (9) (2011) 562–575.